Master 2 CIM - DA - DT 2026/2027
ICOM - Université de Lyon 2

Module Champ d'Application (Angular JS)

Travaux Pratiques 5

Services, déploiements

Objectif: L'objectif de cette séance de TP est de comprendre l'intégration des services dans *Angular* et nous verrons également la partie déploiement sur une plateforme comme *GitHub*.

Partie I: Services

Une autre notion importante est l'utilisation des services. Les services sont essentiels car ils permettent de centraliser à la fois le code et les données, et peuvent être exploités dans différentes parties d'une application (par injection dans le constructeur d'un objet). Dans la suite, nous allons créer un service simple puis l'utiliser dans différents composants.

1.1 Depuis un terminal placé à la racine du projet, générez un service nommé `ActiviteService` dans le répertoire `src/app/services` à l'aide de la commande suivante :

```
$ ng generate service services/activite
```

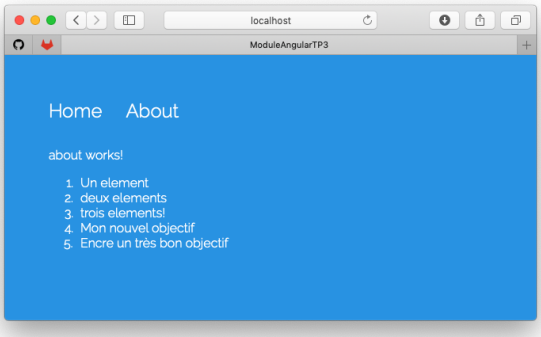
Examinez le fichier `activite.service.ts` créé par Angular. Le décorateur `@Injectable` doit indiquer `providedIn: 'root'` afin que le service soit disponible dans toute l'application. Cette configuration fonctionne aussi bien avec une application utilisant `app.module.ts` qu'avec une application standalone : il n'est alors pas nécessaire d'ajouter le service manuellement dans un tableau `providers`.

1.2 Dans la classe `HomeComponent`, injectez dans le constructeur le service `ActiviteService` en le déclarant comme paramètre et en important correctement la classe.

1.3 Déplacez le tableau `objectifs` de la classe `HomeComponent` vers la classe `ActiviteService`, puis mettez à jour tous les composants qui utilisent ce tableau (y compris dans les templates HTML). Tous les composants doivent accéder au même tableau fourni par le service ; ne créez pas une copie indépendante dans chaque composant. Les animations doivent toujours fonctionner.

1.4 Vérifiez que les composants s'affichent correctement, même après navigation entre *HomeComponent* et *AboutComponent*. Vérifiez également qu'une modification effectuée dans un composant reste visible après la navigation.

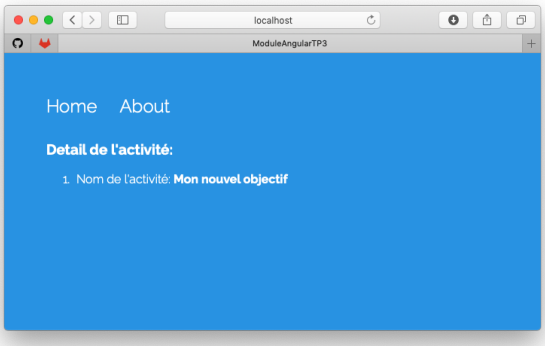
1.5 Reprenez la question **4.3** du TP précédent, mais appliquez cette fois les modifications à la classe `AboutComponent`. Affichez ensuite sous forme d'items la liste des éléments du tableau `objectifs`. Vous pourrez obtenir un affichage similaire à :



Pour terminer, l'objectif des questions suivantes est d'afficher le détail d'une activité. Pour simplifier, les détails se limiteront au titre de l'activité.

1.6 Dans le fichier `home.component.html`, lors de l'affichage des différentes activités, ajoutez un événement souris permettant d'ouvrir la page `consultAct` en ajoutant à l'URL l'indice de l'activité sélectionnée.
Indication : avec la syntaxe de contrôle `@for`, l'indice de l'élément parcouru est accessible avec la variable `$index`. Utilisez cet indice pour construire l'URL. Vérifiez que la route correspondante est bien définie pour recevoir cet indice.

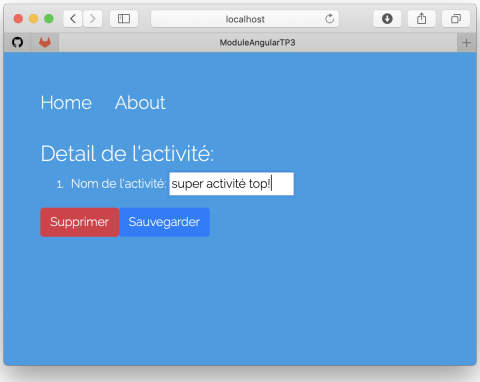
1.7 Dans `consult-activite.component.ts`, injectez le service `ActiviteService`. Récupérez ensuite l'indice transmis dans l'URL à l'aide de `ActivatedRoute` et utilisez-le pour retrouver l'activité correspondante dans le tableau du service. Mettez à jour `consult-activite.component.html` afin d'obtenir un résultat semblable à :

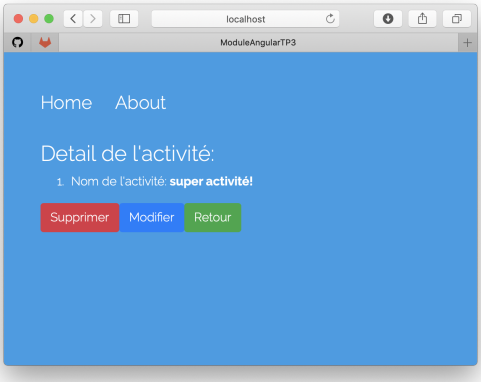


1.8 On souhaite ajouter la possibilité de supprimer un élément de la liste des activités depuis l'onglet « détails ». Pour cela :

- Comme la classe `ActiviteService` centralise le tableau, ajoutez une fonction `removeItem(index: number)` qui supprime l'élément du tableau (utilisez `splice`).
- Dans `ConsultActiviteComponent`, ajoutez une fonction `supprimeElem()` prenant en paramètre l'indice de l'élément à supprimer.
- Ajoutez un bouton permettant d'appeler cette fonction.
- Redirigez ensuite vers la page principale (dans `supprimeElem()`).

1.9 Enfin, on souhaite pouvoir modifier le titre d'une activité. En vous inspirant de la question précédente, ajoutez la fonctionnalité d'édition et de sauvegarde du titre d'une activité, comme illustré ci-dessous :

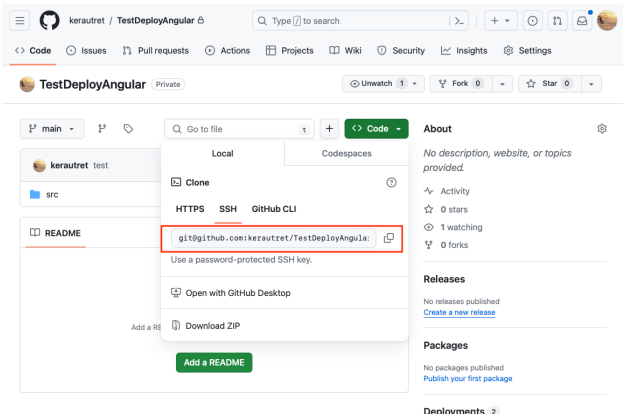




Partie II: Déploiement

Dans cette partie, l'objectif est de déployer en ligne l'application Angular à l'aide de *GitHub Pages*. L'adresse publiée sera de la forme `https://VOTRE_COMPTE.github.io/VOTRE_PROJET/`.
Prérequis : vérifiez que le projet fonctionne localement et que vous êtes placé à la racine du projet dans le terminal. Vous devez également disposer d'un compte GitHub.

2.1 Créez sur *GitHub* un nouveau dépôt vide qui servira à publier votre application. Choisissez un nom simple, sans espace, puis notez le nom du dépôt et votre nom de compte GitHub.



2.2 Associez le projet local au dépôt GitHub. Remplacez les deux éléments en majuscules par les informations de votre compte et de votre dépôt :

```
$ git remote add origin https://github.com/VOTRE\_COMPTE/VOTRE\_PROJET.git
```

Si Git indique qu'un remote nommé `origin` existe déjà, vérifiez son adresse avec `git remote -v`. Si nécessaire, corrigez-la avec :
`git remote set-url origin https://github.com/VOTRE_COMPTE/VOTRE_PROJET.git`.

2.3 Installez l'outil de déploiement si nécessaire. Lors du premier lancement de la commande ci-dessous, Angular peut proposer d'installer le paquet *angular-cli-ghpages*. Acceptez cette installation si elle est proposée ; sinon, ajoutez le paquet en suivant les indications du terminal.

Générez ensuite le déploiement en indiquant le nom du dépôt dans le chemin de référence :

```
$ ng deploy --base-href="/VOTRE\_PROJET/"
```

Attention : conservez les deux barres obliques / autour du nom du dépôt dans l'option `-base-href`.

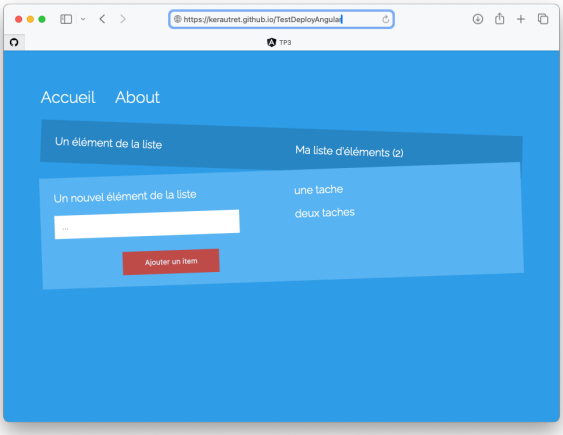
```
kerautre ~/Downloads/CorrectionTP3 ng deploy
Cannot find "deploy" target for the specified project.
You can add a package that implements these capabilities.

For example:
  Amazon S3: ng add @jefiozie/ngx-aws-deploy
  Firebase: ng add @angular/fire
  Netlify: ng add @netlify-builder/deploy
  NPM: ng add ngx-deploy-npm
  GitHub Pages: ng add angular-cli-ghpages

Would you like to add a package with "deploy" capabilities now?
  No
  Amazon S3
  Firebase
  Netlify
  NPM
❯ GitHub Pages
```

2.4 Ouvrez dans un navigateur l'adresse suivante, en remplaçant les éléments en majuscules : `https://VOTRE_COMPTE.github.io/VOTRE_PROJET/`.

Vérifiez que l'application s'affiche correctement et testez ses principales fonctionnalités. La publication peut prendre quelques instants après la commande de déploiement.



En cas de problème : vérifiez le nom du dépôt, la valeur de `-base-href` et les messages affichés dans le terminal. Avec le routage Angular, le rechargement direct d'une page interne peut également nécessiter une configuration supplémentaire sur GitHub Pages ; distinguez ce problème d'un échec général du déploiement.